

# **Property-driven Training and Enforcement of compositional safety-critical systems**

Often a safety critical system is required to satisfy certain conditions on its behaviour. For example, the system may be required to satisfy certain safety properties, such as the system output must not exceed a certain value as it can be unsafe to do so. The system may also be required to satisfy certain liveness properties, such as the system must never stop running. The system may also be required to satisfy certain fairness properties, such as the property that the a particular system must never use more resources than other systems.

In this chapter, we will discuss the design of an autonomous vehicle considering the points discussed above. We will discuss the requirements of the system, the properties that need to be satisfied, the training data that is required, and the enforcement mechanism that is used to ensure that the system behaves correctly. We will particularly focus on the compositional techniques that are used to design the system, and the runtime enforcement techniques that are used to guarantee the system behaviour. We will also discuss the results of the verification, and the lessons learned from the design of the system.

## **6.1 Case Study**

An Autonomous vehicle is a perfect example of a safety critical system that can be designed using the compositional techniques discussed in this thesis. The system is

composed of multiple components that interact with each other to achieve a common goal, i.e. the goal of driving safely on the road. Usually, the system is also composed of multiple sensors that provide information about the environment (the road conditions and the driving conditions), and multiple actuators that allow the system to interact with the environment. The system is also composed of multiple controllers that are responsible for making decisions about how to interact with the environment.

### 6.1.1 Road Layout

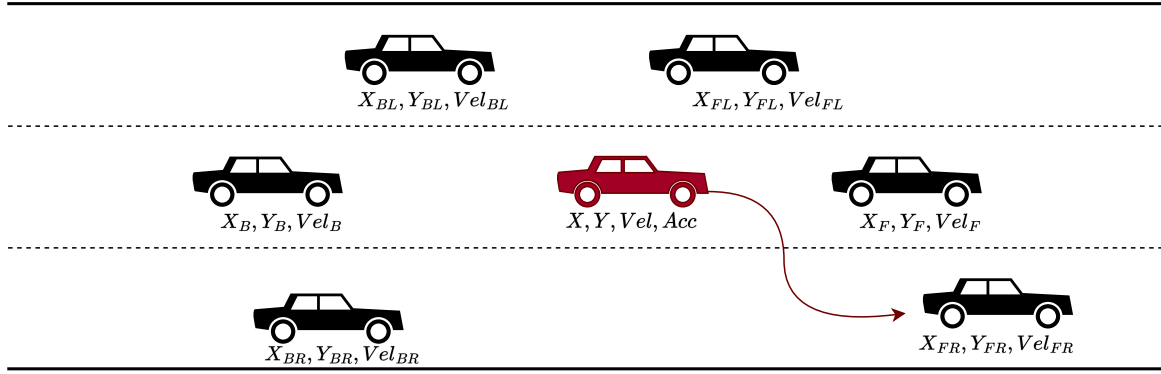


Figure 6.1: General car placement on a freeway

Consider the car placement on a freeway as shown in Figure 6.1. This will be the default car placement for this case study. Specifically, the red car is the subject vehicle and it is surrounded by six other cars at any time - two in either lanes and one in front and one at back. If any of the surrounding cars are missing, an imaginary car at the edge of the sensor range is considered.

Figure 6.2 shows the various modules that control the behaviour of the subject vehicle. We do not consider any specific sensors in this work, but assume that the car has sensors adequate to provide the information required for the AV to make decisions. This is accounted for by the Perception module. The LCD module is responsible for deciding when it is safe to change lanes. The CF and LCE modules are mathematical models that control the car following behaviour, lane change execution, and obstacle avoidance, respectively. The CF module is responsible for maintaining a safe distance from the car in front. The LCE module is responsible for executing the lane change. The controller module controls changes in the car behaviour by choosing which module outputs should be sent to the car controls.

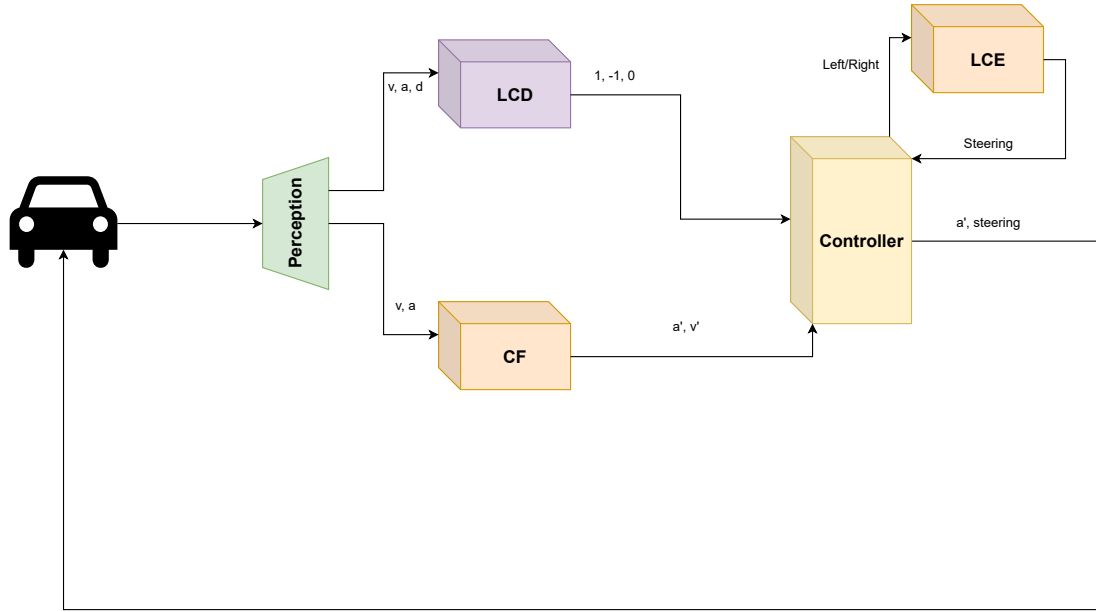


Figure 6.2: AV case study

### 6.1.2 Car Following Module

We use the Intelligent Driver Model (IDM) model for Car Following (CF). The IDM model is a car-following model that describes the dynamics of a vehicle following another vehicle. The model is based on the assumption that the vehicle follows the vehicle in front by maintaining a safe distance. The model is given by the following equation:

$$\begin{aligned} \dot{v} &= a \left( 1 - \left( \frac{v}{v_0} \right)^\delta - \left( \frac{s^*}{s} \right)^2 \right) \\ \dot{s} &= v \end{aligned} \tag{6.1}$$

where  $v$  is the velocity of the vehicle,  $s$  is the distance between the vehicle and the vehicle in front,  $a$  is the acceleration,  $v_0$  is the desired velocity,  $\delta$  is the acceleration exponent, and  $s^*$  is the desired safe distance. We choose  $v_0 = 30m/s$  and  $s^* = 2m$ . IDM model is used in this work for its simplicity and its ability to include driver preferences in modelling car-following behaviour.

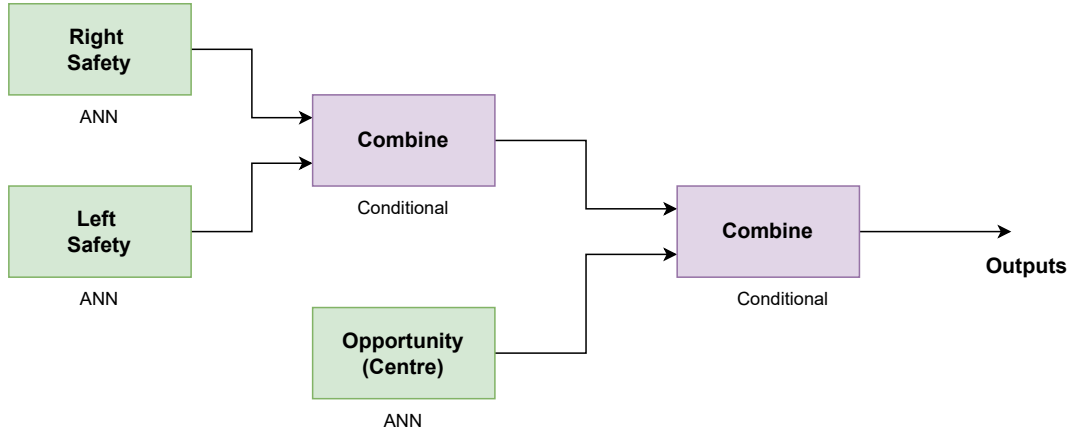


Figure 6.3: 3-stage decomposition of the Lane Change Decision Module

### 6.1.3 Lane Change Decision Module

The Lane Change Decision (LCD) ANN model is responsible for deciding when it is opportune and safe to change lanes. It is a compositional ANN module with three stages as shown in Figure 6.3. The module contains three ANN models and two *merge* blocks. The Left and Right Safety ANN models are responsible for deciding if it is safe to change lanes to the left or right, respectively. The Merge block combines the outputs of the Left and Right Safety ANN models to decide if it is safe to change lanes. The Opportunity block is responsible for deciding if it is opportune to change lanes. The merge block combines the outputs of the Safety and Opportunity blocks to decide if it is safe and opportune to change lanes.

To elaborate further, the Left Safety model takes the left and centre lane car states as inputs, namely, the velocity, acceleration, positions of the five cars (including the subject vehicle) in the left and centre lanes. We also consider the maximum desired velocity of the subject vehicle to consider lane change incentive. Similarly, the Right Safety model takes the right and centre lane car states as inputs. The Opportunity model takes the velocity, acceleration, position, and lane of the cars in the centre lane only as inputs. All three models output a binary variable (0 or 1) as output indicating whether the car should lane change or not. While the first merge block also outputs a binary decision, the final merge block outputs -1, 1, or 0, indicating whether the car should lane change to the left, right or stay in the exiting lane, respectively.

### 6.1.3.1 HighD Dataset

This study uses the highD trajectory dataset recorded by drones at six different German highways ([1]) to build the models for the LCD module. The dataset contains 60 recordings over 16.5 hours from six locations (labelled 1–6) at different times of the day, covering various density of traffic flows, at a frame frequency of 25Hz. The vehicle trajectories were recorded on highways with either no speed limit or with a limit of 120 km/h and 130 km/h. The typical speeds for trucks and cars at the recording sites are 80 km/h and 120 km/h, respectively. The vehicles (81% cars and 19% trucks) in the dataset travel a total of 45,000km, with 11,000 lane changes and 5,600 complete lane changes [1]. Several algorithms and post-processing were applied on the raw trajectory data to retrieve smooth position, velocities, and accelerations in both x and y directions, and information, such as the lane number, the time and space headway among others, about the vehicle trajectories.

The car and truck speeds in highD are closer to the actual highway traffic flow and are much higher than those in the much widely used NGSIM dataset [3]. Moreover, the highD dataset includes more than 11,000 lane changes, which is twice the number of NGSIM. Furthermore, the highD dataset has a higher quality of data than NGSIM. Specifically, the NGSIM dataset is coarser with samples collected at 10 Hz, and it includes many errors in the trajectory of the vehicles performing lane changes [1]. This affects the model parameters and the ability of the model to generalise to actual scenarios. Therefore, we choose the highD dataset for our developing and testing our approach.

### 6.1.4 Lane Change Execution Model

The LCE model is a simple polynomial model that describes the lateral position of the vehicle during a lane change. The model is based on the assumption that the vehicle changes lanes by following a polynomial trajectory. The lateral velocity is computed by taking the derivative of the polynomial that describes the lateral position. The model is given by the following equation:

$$\begin{aligned}
 y(t) &= a_3 t^3 + a_2 t^2 + a_1 t + a_0 \\
 \dot{y}(t) &= 3a_3 t^2 + 2a_2 t + a_1 \\
 a_1 &= 0, a_2 = 0, a_3 = 3 \frac{\text{lane\_width}}{t_{LC}^2}, a_4 = -2 \frac{\text{lane\_width}}{t_{LC}^3} \quad (6.2)
 \end{aligned}$$

where  $y(t)$  is the lateral position of the vehicle at time  $t$ ,  $a_3$ ,  $a_2$ ,  $a_1$ , and  $a_0$  are the polynomial coefficients, and  $t$  is the time.  $t_{LC}$  is the desired time for lane change. Here,  $t_{LC} = 5seconds$  and  $lane\_width = 3.7m$ . The model is used in this work for its simplicity and its ability to describe the lateral position of the vehicle during a lane change.

### 6.1.5 Controller Module

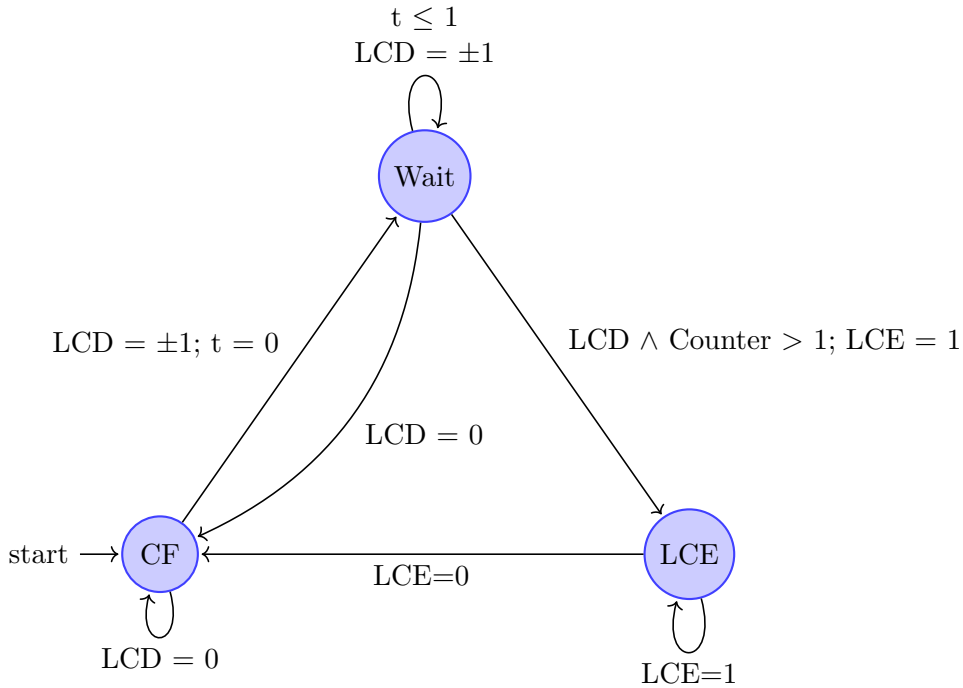


Figure 6.4: A finite state model diagram for the AV controller. Each state represents a phase of operation, and transitions are defined by the conditions derived from inputs and internal counters.

The finite state diagram shown in Figure 6.4 illustrates the operation of the Autonomous Vehicle (AV) controller. The controller switches between three primary states: **Car Following (CF)**, **Waiting (Wait)**, and **Lane Change Execution (LCE)**. Each state corresponds to a phase in the operation, and transitions between these states are governed by specific conditions derived from inputs, time variables, and internal counters.

The system begins in the **Car Following (CF)** state. While in this state, the controller remains in CF as long as the Lane Change Decision (LCD) signal is zero, indicating no decision to perform a lane change. However, when an LCD signal of  $\pm 1$

(indicating a left or right lane change decision) is received, and the timer  $t = 0$ , the system transitions to the **Waiting (Wait)** state. This transition initiates a temporary state where the lane change decision is further evaluated.

In the **Waiting (Wait)** state, the controller assesses whether the LCD signal persists and the timer  $t \leq 1$  second, which keeps the system in the same state. This delay helps ensure that transient LCD signals do not cause premature lane change execution. If the LCD signal continues and an internal counter surpasses a threshold value of 1 second (10 steps at 10 Hertz sampling rate), the controller activates Lane Change Execution (LCE) by transitioning to the **Lane Change Execution (LCE)** state. Conversely, if the LCD signal returns to zero during the Waiting state, the system transitions back to the **Car Following (CF)** state, aborting the lane change decision.

In the **Lane Change Execution (LCE)** state, the controller actively performs a lane change. It remains in this state as long as the LCE signal is set to 1, signifying that the execution is ongoing. Once the LCE signal switches to 0, indicating the completion of the lane change, the controller transitions back to the **Car Following (CF)** state, resuming its default behaviour.

## 6.2 Safety of Autonomous Vehicle

Autonomous vehicle is a good example of a safety critical and time critical system. All the modules in an AV must work in absolute synchronicity to ensure the safety of the vehicle. It is often necessary to provide formal guarantees on the safety of the AV. However, it is often difficult to verify complex systems like an AV that is composed of several interacting components. This problem is further exacerbated by the fact that AVs often use ANNs as controllers (the LCD module in this case study). ANNs are known to be difficult to verify due to their non-linear nature and their large (often infinite) input-output space.

One solution to this problem is to use compositional Assume-Guarantee reasoning. Assume-Guarantee reasoning is a formal method that assumes conditions and guarantees conditions. Moreover, compositional reasoning is a technique that allows us to reason about the system as a composition of its components. This allows us to reason about the system in a modular way, and to verify the system by verifying its components. This is particularly useful in the case of AVs, where the system is composed of several interacting components.

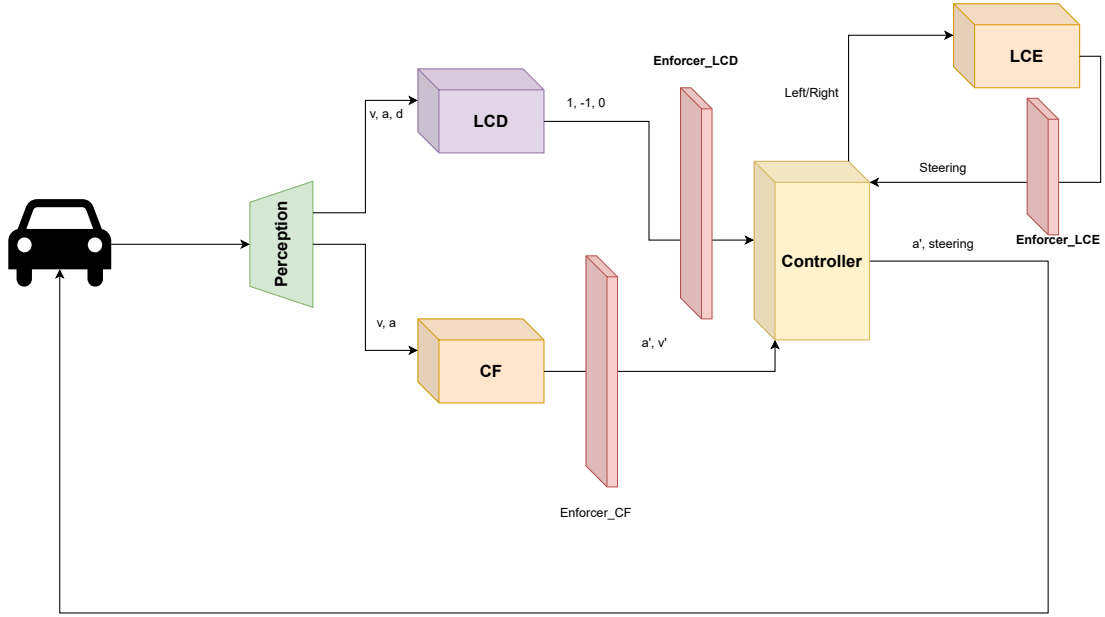


Figure 6.5: AV case study with runtime enforcement

In this case study, we assume the satisfaction of certain properties to guarantee the safety of the AV. To ensure that the properties are satisfied, we do not use formal verification approaches and rather use runtime enforcement techniques. Runtime enforcement is a technique that ensures that the system satisfies certain properties at runtime. It does this by monitoring the system at runtime, and by enforcing certain constraints on the system. If the system violates the constraints, the runtime enforcement mechanism takes corrective actions to bring the system back to a safe state.

We add enforcers at the outputs of all three modules to ensure that the controller gets correct output to be passed onto the car controls. We do not need another enforcer for the controller as the inputs to the controller are being corrected to satisfy safety properties. As long as the controller and the properties are specified correctly, the car should receive safe inputs to work on.

Furthermore, we would like to ensure that the enforcer correction should not happen too often. To achieve this, we also propose a compositional policy-driven training of the models based on a novel policy mining technique. The training ensures that the models adhere to mined policies with certain accuracy. The following sections describe enforcement, the safety policies and the compositional training in detail.

## 6.3 Runtime Enforcement

Using formal runtime monitoring approaches and tools, verification/enforcement monitors (enforcers) can be synthesized from the formal requirements of desired policies, which can be integrated with the system (as shown in Figure 6.6). The enforcer is fed with the current execution of the system. It verifies the current execution with respect to the desired policies and enforces (corrects) the current execution when dealing with the enforcement of the desired safety policies at runtime.

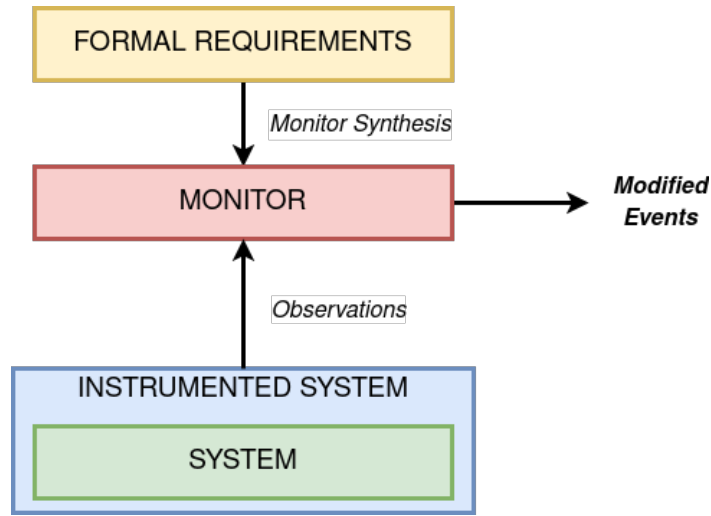


Figure 6.6: An overview of runtime enforcement process

### 6.3.1 Enforcer Synthesis

In our work, we use the enforcer synthesis tools that are based on the approaches proposed in [?] that are suitable for safety-critical and cyber-physical systems. The framework in [?] supports RE monitor synthesis for timed policies. Valued Discrete Timed Automata (VDTA) is used for the formal specification of policies, where valued signals, internal variables, and complex guard conditions are supported, ensuring compatibility with real-world CPS and industrial systems and enforcement monitors are synthesized from these VDTA to enforce a set of desired policies.

*Preliminaries to RE.* A (finite) word is a finite sequence of elements of finite alphabet  $\Sigma$ . The length of  $w$  is the number of elements in  $w$  and is denoted as  $|w|$ . The sets of all words are denoted by  $\Sigma^*$ . A language (property) over  $\Sigma$  is any subset of  $\Sigma^*$  and is denoted by  $\varphi$ .

Let  $\mathbb{R}$  denote the set of real numbers.  $M : \mathbb{R}^n \rightarrow \mathbb{R}^m$  is a function which takes as input a vector of  $n$  real valued numbers and gives as output a vector of  $m$  real valued numbers. This function represents an ANN model.

*Enforcer for  $\varphi$ .* An enforcer (from [? ]) for a property  $\varphi$  is a function  $E_\varphi : \Sigma^* \rightarrow \Sigma^*$  that takes as input a word  $\Sigma^*$  and outputs a word that satisfies  $\varphi$  or can be extended to satisfy  $\varphi$  in the future. It can only edit an input event when necessary, and it can not block, delay or suppress events.

Some constraints are required on how an enforcer transforms words. These constraints describe the expected behaviour of an enforcer and are recalled below (Def. from []).

*Definition 1.* Given property  $\varphi$ , an enforcer for  $\varphi$  satisfies the following constraints:

**Soundness:**

$$\forall \sigma \in \Sigma^*, \exists \sigma_{extn} \in \Sigma^* : E_\varphi(\sigma) \cdot \sigma_{extn} \models \varphi$$

**Monotonicity:**

$$\forall \sigma, \sigma_{extn} \in \Sigma^* : \sigma \preceq \sigma_{extn} \implies E_\varphi(\sigma) \preceq E_\varphi(\sigma_{extn}).$$

**Transparency:**

$$\forall \sigma, a, \sigma_{extn} \in \Sigma^* : E_\varphi(\sigma) \cdot a \cdot \sigma_{extn} \models \varphi \implies E_\varphi(\sigma \cdot a) = E_\varphi(\sigma) \cdot a.$$

*Soundness* means that for any input word, the output of the enforcer can be extended to a sequence that satisfies  $\varphi$ . *Monotonicity* means that the output is a continuously growing word: the output of the enforcer for an extended input word  $\sigma_{extn}$  of an input word  $\sigma$ , extends the output produced by the enforcer for  $\sigma$ . *Transparency*<sup>1</sup> means that the enforcer will not unnecessarily edit any event i.e., for any given input sequence  $\sigma$  and any input event  $a$ , if the output of the enforcer for  $\sigma$  (i.e.,  $E_\varphi(\sigma)$ ) followed by the input event  $a$  has a extension  $\sigma_{extn}$  such that  $E_\varphi(\sigma) \cdot a \cdot \sigma_{extn}$  satisfies the property  $\varphi$ , then the output that the enforcer produces for input  $\sigma \cdot a$  will be  $E_\varphi(\sigma) \cdot a$ .

---

<sup>1</sup>[? ] presents a bidirectional enforcer, whereas our approach focuses on a unidirectional enforcer. Our enforcer corrects inputs from the system in a single direction and is synthesized using the monitor synthesis approach outlined in [? ] only. Consequently, the transparency constraints defined in Def. 1 of our work are slightly modified compared to those in Def. 2 of [? ].

Some other constraints are required, specifically to respect synchronous execution in reactive systems, such as *instantaneity* and is recalled below (Def. from []).

*Definition 2.* Given property  $\varphi$ , an enforcer for  $\varphi$  satisfies the following constraint:

**Instantaneity:**

$$\forall \sigma \in \Sigma^* : |\sigma| = |E_\varphi(\sigma)|$$

*Instantaneity* means that for any given input sequence, the length of the output sequence of the enforcer should be the same as the input sequence. This means that the enforcer cannot delay, insert and suppress events. Whenever the enforcer receives a new input event, it must react instantaneously and produce an output event immediately. This requirement is essential for the enforcement of CPSs, which are reactive in nature.

*Edit Functions.* Given property  $\varphi$ , we introduce  $edit_\varphi$ , which the enforcer uses for editing input events (whenever necessary), to satisfy property  $\varphi$ . This definition is an adapted version of the edit function presented in Sect. 2.2 of []. The adaptation eliminates bidirectionality, resulting in a purely unidirectional enforcer.

*Definition 3* (Edit function). Given an input word  $\sigma \in \Sigma^*$  and an input event  $x \in \Sigma$ ,  $edit_\varphi(\sigma, x)$  is the set of events  $x' \in \Sigma$  such that the word obtained by extending  $\sigma$  with  $x'$  can be extended to a sequence that satisfies the property  $\varphi$  (i.e.,  $\exists \sigma_{extn} \in \Sigma^*$  such that  $\sigma \cdot x' \cdot \sigma_{extn} \models \varphi$ ). Formally,

$$edit_\varphi(\sigma, x) = \{x' \in \Sigma : \exists \sigma_{extn} \in \Sigma^*, \sigma \cdot x' \cdot \sigma_{extn} \models \varphi\}$$

Two proposed methods for editing non-accepting input values are given in []. These are either a random edit, where a random but accepting value would be chosen from a list of accepting values; or a minimum distance edit, where an accepting value would be chosen with minimum binary distance compared to the current non-accepting value.

*Remark* (Soundness, Monotonicity, Transparency and Instantaneity). It has been proved that the enforcer synthesized using the approach outlined in [] satisfies soundness, monotonicity, transparency and instantaneity constraints given in Def ?? and ??.

Since our system is composed of (ML) models connected sequentially or in parallel with an enforcer connected after the entire system, we define properties, such as *inter-model consistency* and *robustness*.

The concept of *inter-model consistency* arises from the observation that, in a decomposed ML system, multiple smaller models  $M_1, \dots, M_k$  are interconnected. The output of one model often serves as the input to the next, forming a sequence of dependencies. The enforcer operates after the entire system, ensuring that the final output satisfies a global property  $\varphi$ . The intuition behind *inter-model consistency* is that the relationships and dependencies between sub-models  $M_1, \dots, M_k$ , are conserved by the enforcer  $E_\varphi$  even after corrections, i.e., their intended role and do not contradict each other after correction.

The concept of *robustness* arises from the observation that an ML model may fail to handle adversarial or unexpected inputs, potentially resulting in unsafe or unreliable behavior. By placing enforcer after the (decomposed) model, the system ensures operation within predefined bounds, even in scenarios that were not covered during training. In essence, an enforcer acts as a runtime safeguard in managing edge cases and adversarial inputs effectively.

*Definition 4.* Given  $n$  decomposed models  $M_1, \dots, M_n$ , given input vector  $x_1, \dots, x_n$  to each model and given an enforcer  $E_\varphi$  for property  $\varphi$ , placed after the entire decomposed model, whose final output is obtained by  $E^{out}$ . The system satisfies the following constraint:

**Inter-model consistency:**

$$E^{out} \models \varphi \implies \left( \forall M_j : M_i \rightarrow M_j \implies M_j(M_i^{corr}(x_i^{corr})) \sim M_j(M_i(x_i)) \right)$$

**Robustness:**

$$\forall M_i \in \{M_1, \dots, M_n\}, \forall \sigma_{adv} \in \Sigma^* \left( M_i(\sigma_{adv}) \implies E^{out} \models \varphi \right)$$

*Inter-model consistency* expresses that if  $M_i$  is corrected by the enforcer to produce  $M_i^{corr}(x_i^{corr})$ , given any model  $M_j$  that depends on  $M_i$ 's output must still behave compatibly with how it would have behaved with the original, uncorrected output  $M_i(x_i)$  provided the final enforced output  $E^{out}$  satisfies property  $\varphi$ . It means that the relationships and dependencies between sub-models  $M_1, \dots, M_k$ , are conserved by the enforcer even after corrections. Here,  $\sim$  denotes compatibility, i.e.,  $M_j(M_i^{corr}(x_i^{corr})) \sim M_j(M_i(x_i))$  if they belong to same valid input space.

*Robustness* means that for every model  $M_i$  and every adversarial input  $\sigma_{adv}$ , the enforcer ensures that even if  $M_i$  produces an incorrect or unsafe result due to  $\sigma_{adv}$ , the final output of the enforcer  $E^{out}$  will satisfy the required property  $\varphi$ . In simpler

terms, no matter how an adversarial input impacts the models, the enforcer ensures the system adheres to the property.

*Proposition 1* (Inter-model consistency, Robustness). Given a property  $\varphi$ , its enforcer  $E_\varphi$  satisfies Inter-model consistency and Robustness constraints as per Def. 4.

## 6.4 Policies

### 6.4.1 System-Level Property

The system-level property for the AV case study we consider is very simple. We want to ensure that the AV does not collide with any other vehicle on the road. This is a very basic safety property that is required for any AV. The property can be decomposed into three sub-properties, one for each module.

- $P_{LC}$ : No Collision Due to Lane Change
  - $P_{LCD}$ : No collision due to unsafe lane change decision
    - \*  $P_{LF}$ : No collision with the left front car due to unsafe lane change to the left
    - \*  $P_{LP}$ : No collision with the left rear car due to unsafe lane change to the left
    - \*  $P_{RF}$ : No collision with the right front car due to unsafe lane change to the right
    - \*  $P_{RP}$ : No collision with the right rear car due to unsafe lane change to the right
    - \*  $P_F$ : No collision with the front car due to unsafe lane change
    - \*  $P_P$ : No collision with the back car due to unsafe lane change
  - $P_{LCE}$ : No collision due to lane change execution taking too long to finish
- $P_{CF}$ : No collision with front car due to Car following

The composition of these properties ensures that the AV does not collide with any other vehicle on the road.

$$\begin{aligned}
 P_{LC} &= P_{LCD} \wedge P_{LCE} \\
 P_{LCD} &= ((P_{LF} \wedge P_{LP}) \vee (P_{RF} \wedge P_{RP})) \wedge (P_F \wedge P_P) \\
 P_{safety} &= P_{LC} \wedge P_{CF}
 \end{aligned} \tag{6.3}$$

As the goal here is to just show the compositional policy mining and training, we only focus on three safety properties ( $P_{LCE}, P_{LCD}, PCF$ ), one for each module. More properties can be defined, but the composition will be similar. Following subsections define the properties in detail as an FSM and the enforcement mechanism to ensure that the properties are satisfied at runtime.

### 6.4.2 Safety Property for Discretionary Lane Change

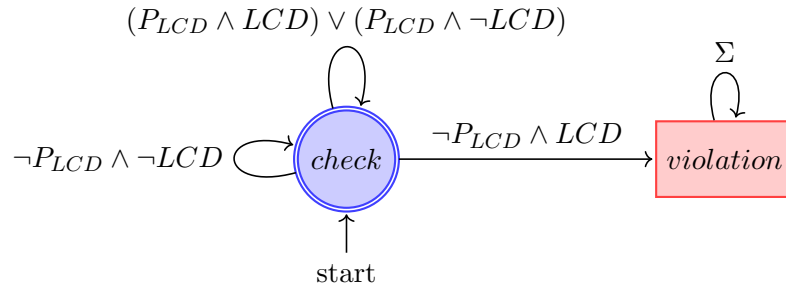


Figure 6.7: Safety property for discretionary lane change module

Here,  $P_{LCD} = ((P_{LF} \wedge P_{LP}) \vee (P_{RF} \wedge P_{RP})) \wedge (P_F \wedge P_P)$ . If  $P = 1$ , it is considered safe to perform a lane change, while if  $P = 0$ , it is not safe. The variable  $LCD$  represents the output of the Lane Change Decision (LCD) module, where  $LCD = 1$  indicates a decision to change lanes, and  $LCD = 0$  indicates no lane change.

The safety property illustrated in Figure 6.7 ensures that a lane change decision is only made when it is deemed safe by the property  $P$ . The transitions can be summarized as follows:

Safe Operation: The system remains in the initial state *check* as long as:

- $P_{LCD} \wedge LCD$ : The LCD module outputs a lane change decision, and the safety property  $P$  indicates it is safe to change lanes.
- $P_{LCD} \wedge \neg LCD$ : The safety property  $P$  indicates it is safe, but the LCD module outputs no lane change.
- $\neg P_{LCD} \wedge \neg LCD$ : The LCD module outputs no lane change, and the safety property  $P$  indicates it is unsafe to change lanes.

These conditions ensure that the system operates safely and avoids unnecessary violations.

Violation State: The system transitions to the violation state *violation* if  $\neg P_{LCD} \wedge LCD$ , meaning the LCD module outputs a lane change decision while the safety

property  $P_{LCD}$  indicates it is unsafe to do so. This represents a critical safety violation where the system attempts to change lanes in unsafe conditions.

**Violation Correction:** The Enforcer corrects the violation by making the  $LCD$  output 0, preventing the unsafe lane change.

This safety property guarantees that the Lane Change Decision (LCD) module operates within defined safety constraints, preventing unsafe lane changes as long the policies are defined correctly.

### 6.4.3 Safety Property for Car Following

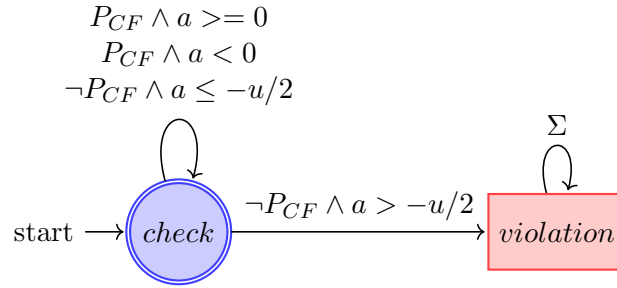


Figure 6.8: Safety property for car following module

Figure 6.8 shows the safety property for the Car Following (CF) module. The property ensures that the car maintains safe behavior during car-following scenarios based on the conditions of acceleration ( $a$ ), speed ( $u$ ), and gap to the vehicle in front.

Here,  $P_{CF}$  is a property indicating whether the car is safely following the car ahead. For instance,  $P_{CF} = 1$  indicates the car is following safely, while  $P_{CF} = 0$  indicates the car is not following safely. The property operates as follows:

**Safe Operation:** The system remains in the initial state *check* under the following conditions:

- $P_{CF} \wedge a \geq 0$ : The car is following safely, and its acceleration is non-negative, meaning it is either maintaining or increasing its speed.
- $P_{CF} \wedge a < 0$ : The car is following safely, but its acceleration is negative, indicating it is slowing down.
- $\neg P_{CF} \wedge a \leq -\frac{u^2}{2 \cdot gap}$ : The car is not following safely, but it is decelerating sufficiently to avoid a potential collision. Here, the term  $-\frac{u^2}{2 \cdot gap}$  represents the critical deceleration needed to ensure a safe stop within the available gap.

**Violation State:** The system transitions to the *violation* state if  $\neg P_{CF} \wedge a > -\frac{u^2}{2 \cdot gap}$ , meaning the car is not following safely, and its deceleration is insufficient to avoid a collision. This represents a safety-critical situation where corrective action is needed.

**Violation Correction:** The Enforcer corrects the violation by making  $a = -u^2/(2 * gap)$  or by braking.

This property ensures that the Car Following (CF) module adheres to safe car-following dynamics, preventing potential collisions by appropriately monitoring and controlling acceleration and maintaining a safe distance.

#### 6.4.4 Safety Property for Lane Change Execution

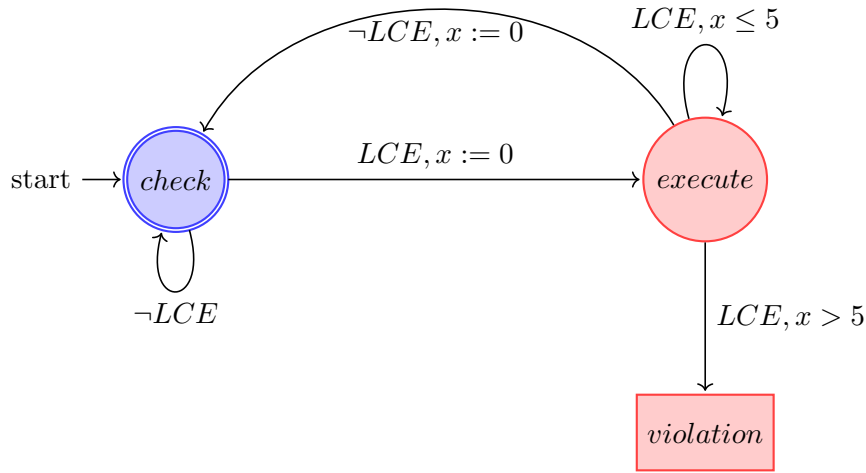


Figure 6.9: Safety property for the lane change execution module

Figure 6.9 illustrates the safety property for the Lane Change Execution (LCE) module. This property ensures that the lane change is executed within a predefined time threshold, guaranteeing adherence to safety requirements and preventing violations.

In this state diagram,  $LCE$  represents the Lane Change Execution signal, where  $LCE = 1$  indicates an ongoing lane change, and  $LCE = 0$  indicates no active lane change. The variable  $x$  tracks the duration of the lane change. The system's behavior is described as follows:

**Safe Operation:** The system starts in the *check* state and remains there under the following conditions:

- $\neg LCE$ : No lane change is being executed ( $LCE = 0$ ), allowing the system to remain in the safe *check* state. This is represented by a self-loop on the *check* state.
- $LCE \wedge x := 0$ : If a lane change is initiated ( $LCE = 1$ ), the system transitions to the *execute* state, resetting the duration variable  $x$  to zero.

**Lane Change Execution:** The system enters the *execute* state when  $LCE = 1$ . While in this state:

- $LCE \wedge x \leq 5$ : The lane change continues safely as long as the duration  $x$  does not exceed the threshold of 5 seconds. This is represented by a self-loop on the *execute* state.
- $\neg LCE; x := 0$ : If the lane change is terminated ( $LCE = 0$ ), the system transitions back to the *check* state, resetting the duration variable  $x$ .

**Violation State:** The system transitions to the *violation* state when  $LCE \wedge x > 5$ , indicating that the lane change duration exceeds the safe threshold of 5 seconds. This represents a critical safety violation, as prolonged lane change execution can compromise safe driving conditions.

**Violation Correction:** The system can only recover from the *violation* state by terminating the lane change execution ( $LCE = 0$ ), ensuring that the violation is resolved and the system returns to a safe state.

This property ensures that the Lane Change Execution (LCE) module operates safely by enforcing a time constraint on the lane change process, thus preventing unsafe or prolonged lane changes.

## 6.5 Policy-based Compositional Training

We develop a novel compositional policy mining and training technique to train the models in the LCD module. The algorithm provides a systematic approach to ensuring that models, particularly in safety-critical applications like autonomous driving, satisfy desired properties with certain accuracy. Note that although the technique presented discusses compositional policies and models, the algorithm is model agnostic and can be modified easily to accommodate monolithic (single) properties and models.

By leveraging Ordinary Least Squares for initial coefficient estimation and BFGS for optimization, the approach ensures that the model parameters align with safety requirements, while the filtering and model merging steps guarantee that the final

system adheres to the overall safety properties. As both the objective function and the optimisation function (MSE) are both convex functions, the whole optimisation is convex. However, as we are stopping early, there is a good possibility of the solution being non-optimal. We stop early to ensure that the properties obtained do not overfit to the data.

To elaborate, the algorithm is divided into two parts: initialization and optimization, and filtering and training. The initialization and optimization part focuses on optimizing the coefficients of the models to satisfy the safety properties. The filtering and training part focuses on filtering the data based on the satisfaction of the safety properties and training the models on the filtered data. The algorithm ensures that the models satisfy the safety properties and maximize the Accuracy of the system. The algorithm is outlined in Algorithms 5 and 6. Following is a detailed explanation of each step in the algorithm.

## 6.5.1 Training Algorithm

### 6.5.1.1 Property Initialization

The algorithm begins by defining a property template  $P$ , represented as an inequality involving a threshold  $\mathcal{A}$  and a linear combination of features weighted by coefficients  $\mathbf{B}$ . Formally, the property is expressed as:

$$P : \mathcal{A} - (\mathbf{X}_i \mathbf{B}) \geq 0$$

where  $\mathbf{X}_i$  represents the features of the dataset sample  $i$ ,  $\mathcal{A}$  is a fixed threshold, and  $\mathbf{B}$  are the coefficients to optimize. This formulation enables encoding constraints or safety properties into a mathematical framework, ensuring that samples satisfying the inequality adhere to the desired property.

### 6.5.1.2 Initial Coefficient Estimation via Ordinary Least Squares (OLS)

To initialize the optimization process, the algorithm estimates the coefficients  $\mathbf{B}$  using Ordinary Least Squares (OLS) regression. This involves minimizing the squared error between the observed values  $y_i$  and the predicted values  $\mathbf{X}_i \mathbf{B}$ , calculated as:

$$\hat{\mathbf{B}} = \arg \min_{\mathbf{B}} \sum_{i=1}^N (y_i - \mathbf{X}_i \mathbf{B})^2$$

---

**Algorithm 5** Property-Based Training with Convex Optimization (Part 1: Initialization and Optimization)

---

- 1: **Input:** Dataset  $\mathcal{D} = \{(\mathbf{X}_i, y_i)\}_{i=1}^N$ , where  $\mathbf{X}_i$  are features and  $y_i$  are labels. Target satisfaction percentage  $\mathcal{T}$ , maximum iterations  $\mathcal{I}$ .
- 2: **Output:** Optimized coefficients  $\tilde{x}_1, \dots, \tilde{x}_k$ , satisfaction percentage  $\mathcal{S}$ , and indices  $\mathcal{I}_{\text{sat}}$  of satisfying samples.
- 3: **Step 1: Property Definition**
- 4: Define the property template as an inequality:

$$P : \mathcal{A} - (\mathbf{X}_i \mathbf{B}) \geq 0$$

where  $\mathcal{A}$  is a threshold and  $\mathbf{B}$  are coefficients to optimize.

- 5: **Step 2: Initial Coefficient Estimation via Ordinary Least Squares (OLS)**
- 6: Fit an OLS model using  $\mathcal{D}$  to obtain initial coefficients  $\hat{\mathbf{B}}$ :

$$\hat{\mathbf{B}} = \arg \min_{\mathbf{B}} \sum_{i=1}^N (y_i - \mathbf{X}_i \mathbf{B})^2$$

- 7: **Step 3: Optimization**
- 8: Set the initial threshold  $\mathcal{A}_{\text{fixed}}$  as:

$$\mathcal{A}_{\text{fixed}} = \sum_{j=1}^k \hat{B}_j \cdot \text{mean}(\mathbf{X}_j) - \min(y)$$

- 9: Define the objective function  $\mathcal{F}(\mathbf{B})$ :

$$\mathcal{F}(\mathbf{B}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(P \text{ is satisfied for } (\mathbf{X}_i, y_i))$$

- 10: Perform optimization using BFGS:

$$\tilde{\mathbf{B}} = \arg \min_{\mathbf{B}} \left( \sum_{i=1}^N (y_i - (\mathbf{X}_i \mathbf{B} \pm \mathcal{A}_{\text{fixed}}))^2 \right)$$

Stop early if  $\mathcal{F}(\tilde{\mathbf{B}}) \geq \mathcal{T}$ .

- 11: Return  $\tilde{\mathbf{B}}$ ,  $\mathcal{F}(\tilde{\mathbf{B}})$ , and satisfying indices  $\mathcal{I}_{\text{sat}}$ .
- 

The initial coefficients  $\hat{\mathbf{B}}$  serve as a starting point for subsequent optimization. The threshold  $\mathcal{A}_{\text{fixed}}$  is also initialized based on the fitted coefficients and the dataset characteristics, ensuring that it is aligned with the observed data distribution.

---

**Algorithm 6** Property-Based Training with Convex Optimization (Part 2: Filtering and Training)

---

- 1: **Input:** Dataset  $\mathcal{D}$ , optimized coefficients  $\tilde{\mathbf{B}}$ , satisfying indices  $\mathcal{I}_{\text{sat}}$ , safety property  $P_{\text{safety}}$ .
- 2: **Output:** Trained model  $M$  and final Accuracy  $\mathcal{R}$ .
- 3: **Step 4: Data Filtering**
- 4: Partition  $\mathcal{D}$  into subsets based on satisfaction:

$$\mathcal{D}_{\text{sat}} = \{(\mathbf{X}_i, y_i) \in \mathcal{D} \mid P_{\text{safety}} \text{ is satisfied}\}$$

$$\mathcal{D}_{\text{unsat}} = \{(\mathbf{X}_i, y_i) \in \mathcal{D} \mid \neg P_{\text{safety}} \text{ is satisfied}\}$$

- 5: Retain lane-change samples ( $y_i = 1$  or  $y_i = -1$ ) in  $\mathcal{D}_{\text{sat}}$  and no-lane-change samples ( $y_i = 0$ ) in  $\mathcal{D}_{\text{unsat}}$ .
- 6: **Step 5: Model Training on Filtered Data**
- 7: Merge  $\mathcal{D}_{\text{sat}}$  and  $\mathcal{D}_{\text{unsat}}$  to form the combined dataset  $\mathcal{D}_{\text{combined}}$ .
- 8: Train the model  $M$ :

$$M = \arg \min_{\theta} \sum_{(\mathbf{X}_i, y_i) \in \mathcal{D}_{\text{combined}}} \mathcal{L}(M(\mathbf{X}_i; \theta), y_i)$$

where  $\mathcal{L}$  is the loss function.

State Evaluate the Accuracy  $\mathcal{R}$ :

$$\mathcal{R} = \frac{|\{(\mathbf{X}_i, y_i) \in \mathcal{D}_{\text{combined}} \mid P_{\text{safety}} \text{ is satisfied}\}|}{|\mathcal{D}_{\text{combined}}|}$$

- 9: Update  $M$  to maximize  $\mathcal{R}$  until convergence.
  - 10: Return  $M$  and  $\mathcal{R}$ .
- 

### 6.5.1.3 Constrained Optimization with BFGS

Following the initialization, the algorithm refines the coefficients  $\mathbf{B}$  using the Broyden-Fletcher-Goldfarb-Shanno (BFGS) method, a quasi-Newton optimization approach. The objective function  $\mathcal{F}(\mathbf{B})$  is defined as the proportion of samples satisfying the property  $P$ :

$$\mathcal{F}(\mathbf{B}) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathcal{A} - (\mathbf{X}_i \mathbf{B}) \geq 0)$$

where  $\mathbf{1}$  is the indicator function. The optimization aims to maximize  $\mathcal{F}(\mathbf{B})$  while minimizing the prediction error. The BFGS algorithm (Broyden-Fletcher-Goldfarb-Shanno) is a quasi-Newton method used for optimization. It is particularly effective when the objective function is smooth and continuously differentiable. The BFGS

method iteratively approximates the inverse Hessian matrix to update the solution and improve convergence efficiency. Optimization terminates early if the satisfaction percentage  $\mathcal{F}(\tilde{\mathbf{B}})$  meets or exceeds the target  $\mathcal{T}$ .

#### 6.5.1.4 Data Filtering and Composite Property Formation

After performing the optimization, the dataset is filtered based on the satisfaction of the safety property  $P_{safety}$ . We partition the dataset into two subsets: those samples that satisfy the property  $P_{safety}$  and those that do not. In other words, we combine This composite property is used to create a combined dataset  $\mathcal{D}_{combined}$  for training the final models.

#### 6.5.1.5 Model Training on Property-Satisfying Data

Once the data is filtered and adjusted for property satisfaction, the models are trained using only the property-satisfying samples. This ensures that the models are aligned with the desired safety properties. The objective function for training is the standard loss function:

$$M_i = \arg \min_{\theta_i} \sum_{(\mathbf{X}_j, y_j) \in \mathcal{D}_{combined}} \mathcal{L}(M_i(\mathbf{X}_j; \theta_i), y_j)$$

where  $\mathcal{L}$  is the loss function and  $\theta_i$  represents the model parameters.

Finally, the Accuracy  $\mathcal{R}_i$  of the trained models is evaluated based on their ability to satisfy the property  $P_i$ . The goal is to maximize the Accuracy, which is the proportion of samples for which the model satisfies the property:

$$\mathcal{R}_i = \frac{|\{(\mathbf{X}_j, y_j) \in \mathcal{D}_{combined} \mid P_i \text{ is satisfied by } M_i\}|}{|\mathcal{D}_{combined}|}$$

The model is then updated to maximize this value.

#### 6.5.1.6 Model Merging

After training and optimizing each individual model, the final step involves merging the models into a single merged model  $M_{merged}$ , which ensures that all the properties  $P_1, \dots, P_n$  are satisfied. The merging process combines the outputs of the models such that the final model satisfies the combined set of properties, ensuring safe behavior in the autonomous system.

$$M_{merged} = f(M_1, M_2, \dots, M_n)$$

where  $f$  is a function designed to combine the models while maintaining the property satisfaction criteria.

### 6.5.2 LCD Module Training

We demonstrate the use of the aforementioned policy-based training algorithm to train the models in the Lane Change Decision (LCD) module of the autonomous vehicle. We develop a compositional LCD module consisting of multiple ANN models and also a monolithic LCD module for comparative analysis. Both module designs are trained on data using the policy-based training algorithm. Moreover, we also develop both compositional and monolithic LCD modules that are trained using a standard supervised learning approach without the policy-based training for further comparative examination.

#### Compositional LCD Module:

The LCD module used in this work is a compositional module with several ANN models connected together. Each model in the compositional LCD module is trained on the data satisfying the overall safety property ( $P_{LCD}$ ) separately. The safety property is obtained by mining the individual components of the safety property using the algorithm described in the previous section. The safety policies are of the following form:

$$safe\_gap > constant + rel\_vel * coefficient_1 + rel\_acc * coefficient_2^2 \quad (6.4)$$

For instance,  $P_{LF} : rfx > x0 + x1 * rfxVelocity + x2 * rfxAcceleration$ , where  $rfx$  is the right front car position,  $rfxVelocity$  is the right front car velocity,  $rfxAcceleration$  is the right front car acceleration, and  $x0$  (intercept),  $x1$  and  $x2$  are the coefficients to be learnt/optimised.

The values of the position, velocity and acceleration are obtained from the HighD dataset. We extract datapoints from lane changing and not lane changing vehicles alike. For lane changing vehicles we take the points from a 2 second period around the time when the lane change begins it normally takes 1-2 seconds for a human to gauge the surrounding conditions making a decision to lane change. We gather 2 second regions from not lane changing cars as well as examples of conditions where the car should not decide to lane change. This gives us a total of 28 features - 4 for the ego car, and 4 each for the six surrounding cars. These features include the relative position, velocity, and acceleration of the surrounding cars with respect to the ego car and the

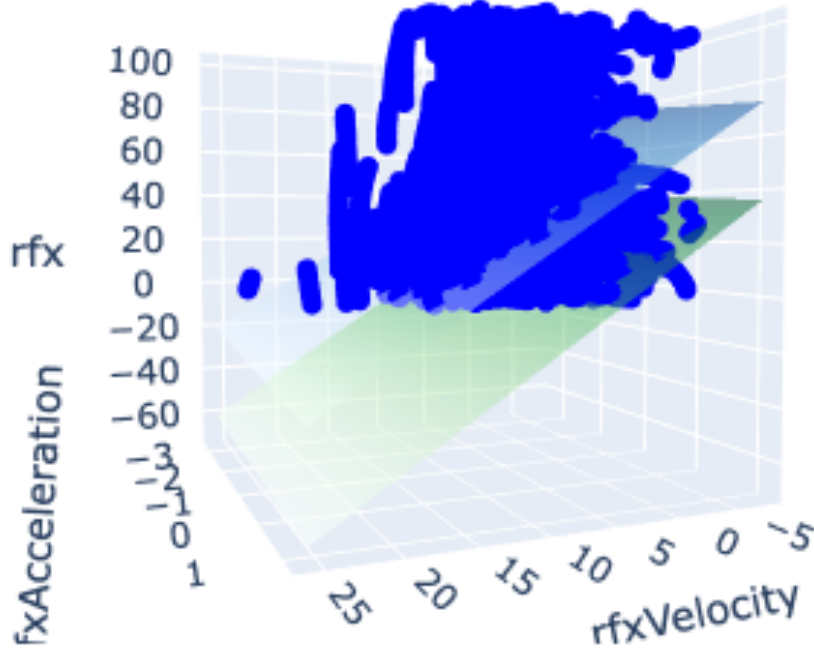


Figure 6.10: Optimisation Planes for the LCD module Property  $P_{RF}$

ego car's own velocities, acceleration and maximum velocity (used as a substitute for desired velocity).

These extracted datapoints are used as the data to mine the properties for the models. Figure 6.10 shows the optimisation planes for the property  $P_{RF}$ . The blue plane shows the plane obtained by Ordinary Least Squares. This is then optimised to the green plane using the BFGS algorithm. The green plane is the plane for which 90% of the datapoints satisfy  $P_{RF}$ . As all the datapoints are from safe lane changes from real cars, the goal is ensure that most points (at least 90%) are above the plane.

The data satisfying the mined property ( $P_{LCD} = ((P_{LF} \wedge P_{LP}) \vee (P_{RF} \wedge P_{RP})) \wedge (P_F \wedge P_P)$ ) are labelled 1 or lane change datapoints and the ones that do not satisfy the property from the no lane change samples are labelled 0 or no lane change datapoints

Table 6.1: Number of datapoints obtained for each satisfaction percentage

| Property Satisfaction | Left LC Data-points | Right LC Data-points | No LC Data-points |
|-----------------------|---------------------|----------------------|-------------------|
| 90%                   | 69,422              | 61,342               | 43,667            |
| 80%                   | 45,998              | 49,866               | 122,521           |
| 70%                   | 42,484              | 40,514               | 128,662           |
| OLS                   | 11,178              | 3,561                | 314,750           |
| No Policy             | 86,940              | 78,133               | 165,073           |

(any points far away from the lane change points are chosen as the no lane change [unsafe lane change] samples).

We also experiment with different target satisfaction percentages ( $\mathcal{T}$ ) to obtain the property-satisfying dataset. We experiment with 70%, 80%, 90% and also consider the case where we just use the coefficients obtained after the OLS step in Algorithm 5 (i.e. we stop at the blue plane in Figure 6.10). The number of datapoints obtained for each case is shown in Table 6.1. We do not try 100% threshold as that leaves very few datapoints for the no lane change class.

All compositional ANN models designed for the LCD module have the same 4-layer architecture with 64, 32, 16, 1 neurons in the layers from inputs to output. Once we divide the 28 input features across the 3 models in the compositional LCD module, we get 20 features each for the left and right lane change decision models, and 12 features for the opportunity model.

Furthermore, the merge blocks are designed such that they help satisfy the combination of individual properties in agreement with the combined lane change property  $P_{LC}$ . The first merge block simply takes the maximum of the two model outputs ensuring that only one of the two models outputs passed through (an *Or* operation). The second merge block takes the output of the first merge block and the output of the opportunity model and merge them using an *And* operation. This ensures that the lane change is only executed when both the safety and opportunity conditions are satisfied. The model performance (accuracy) reported is the accuracy of the whole module obtained by checking the outputs of the second merge block with the actual outputs.

Also, all layers use *tanh* as the activation function for all hidden layers. The output layer uses sigmoid as the activation function. We use Adam optimiser with Binary Crossentropy as the loss function. The models are trained using the 5-fold cross validation approach for 200 epochs with a batch size of 128 using Tensorflow in

Python. The data is oversampled to handle the imbalance (see Table 6.1), but the oversampling is done only on the training data and not on the test data.

The other LCD module is trained using the same architecture and optimisation parameters on the whole dataset without property-driven filtering for comparison (*No Policy* in Table 6.1). We do, however, ensure that the test data from the filtered property-satisfying dataset is not used for training the normal supervised learning model for accurate comparison.

#### **Monolithic LCD Module:**

The monolithic LCD module is a single ANN model with 28 input features and 3 outputs. The model has a 4-layer architecture with 128, 64, 32, 3 neurons in the layers from input to output. The model uses *tanh* as the activation function for all hidden layers and *softmax* as the activation function for the output layer. The model is trained for the same set of datasets as mentioned in Table 6.1 and use the same optimisation parameters as the compositional models.

## **6.6 Simulation**

Once all the properties and modules are defined, the next step is to simulate the behavior of the autonomous vehicle in a controlled environment. The simulation provides a way to observe how the vehicle interacts with its surroundings, makes decisions, and executes actions based on the defined modules and properties. The enforcers are built in C using the easyRTE tool, while the simulation is run in Python as the models are developed in Python. The C code is execute through Python system calls. Figure 6.11 shows the execution flow. At each time step, the enforcer checks the output of the module and corrects it if it violates the safety property. The corrected output is then passed to the controller, which in turn sends the corrected output to the car controls. The corrected output is then used to update the car's position and velocity.

### **6.6.1 Simulation Algorithm**

To elaborate, the simulation operates in a synchronous, tick-based manner, where at each time step, or "tick," the entire system is updated simultaneously. Each tick represents a discrete time interval, and at every tick, the state of all vehicles in the simulation is updated at the same time. The current time is incremented by the time step  $dt$ , which is a constant value that determines the resolution of the simulation.

---

**Algorithm 7** Autonomous Vehicle Simulation and Lane Change Model

---

```
1: Input: Simulation duration sim_duration_sec, time step dt, number of lanes  
   num_lanes, list of cars  $\mathcal{C}$ , models for lane change and IDM.  
2: Output: History of car positions and velocities over time.  
3: Step 1: Initialization  
4: Initialize the IDM model and lane change model.  
5: Initialize controller for lane change execution.  
6: Initialize the cars with their positions, velocities, and lane assignments.  
7: Load pre-trained models for left, right, and center lane behaviors.  
8: Step 2: Simulation Loop  
9: for each time step step in the range num_steps do  
10:   current_time  $\leftarrow$  step  $\times$  dt  
11:   Step 2.1: Data Collection and Processing  
12:   Collect data for left, right, and center lanes using the function get_data( $\mathcal{C}$ ).  
13:   Extract gap and lateral velocity information for ego car.  
14:   Step 2.2: Lane Change Decision  
15:   Compute lane change decision and prediction ( $P$ ,  $N$ ) with the collected data  
   and pre-trained models.  
16:   Pass data to enforcer and pass corrected (if needed) data to controller.  
17:   Use the controller to process the inputs and determine lateral velocity setpoint.  
18:   Step 2.3: Lane Change Decision  
19:   Compute the car following behavior using the IDM model.  
20:   Pass data to enforcer and pass corrected (if needed) data to controller.  
21:   Step 2.4: Lane Change Execution  
22:   Run lane change execution.  
23:   Pass data to enforcer and pass corrected (if needed) data to controller.  
24:   Step 2.5: Update Vehicle Dynamics  
25:   Update ego car's lateral velocity and acceleration based on lane change and  
   gap.  
26:   Update longitudinal motion of all cars.  
27:   Update lateral motion of cars.  
28:   Step 2.6: Lane Change Finalization  
29:   if car finishes lane change then  
30:     Finalize the lane change and update the lane index and positions for the  
     affected cars.  
31:   end if  
32:   Step 2.7: Store History  
33:   Append the current positions and velocities of each car to the history dictio-  
   nary.  
34: end for  
35: Step 3: Return Results  
36: Return the simulation history of all cars, including longitudinal and lateral posi-  
   tions, velocities.
```

---

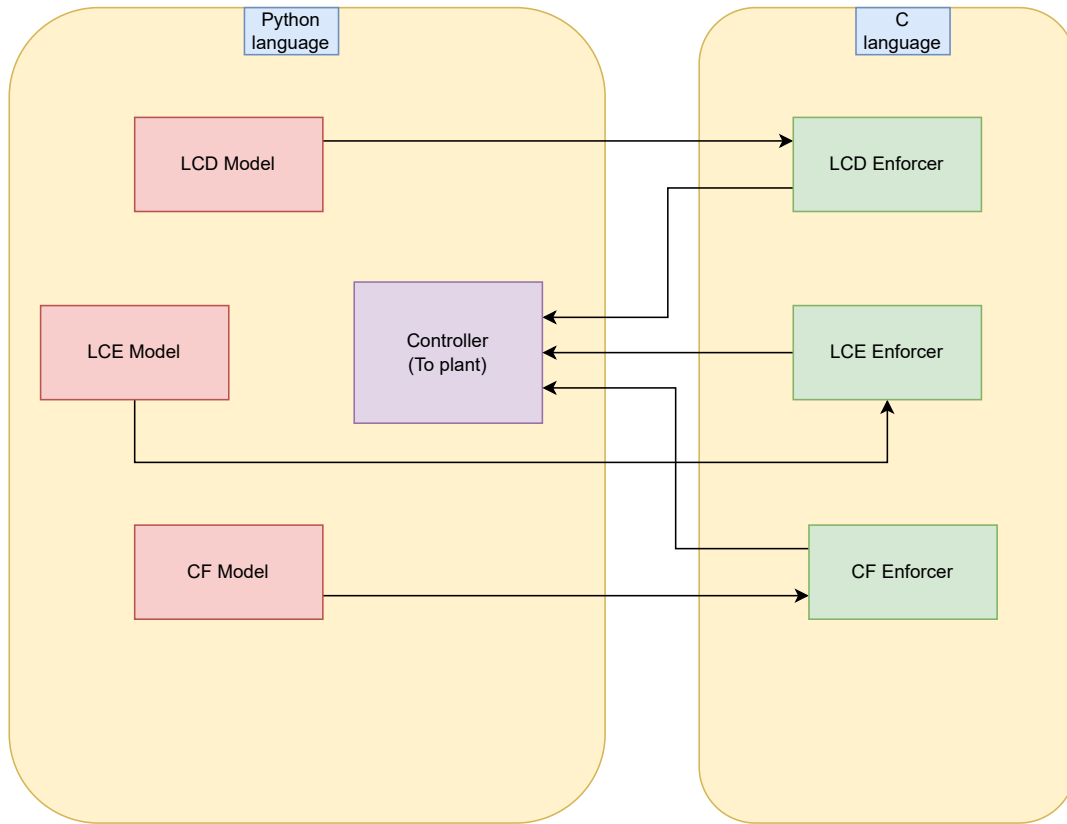


Figure 6.11: Policy Implementation

This means that all vehicles are updated in parallel, and no vehicle progresses ahead or behind others in the simulation timeline.

At the start of each tick, data for every vehicle, such as its position, velocity, and other relevant parameters, is collected. This data is then used by the models, such as the lane change and Intelligent Driver Model (IDM), to compute the necessary actions for the current tick. These actions include deciding whether to change lanes, adjusting velocities, and updating positions. Importantly, all vehicles make these decisions simultaneously, based on their current states and the states of other vehicles in the environment.

Once a lane change decision is made, if applicable, the vehicle's lateral velocity and position are adjusted to execute the change. This happens for all vehicles that decide to change lanes during the current tick. Afterward, the vehicle dynamics are updated, meaning that the position of each vehicle is recalculated based on its velocity, considering any car-following behavior or lane changes that might have occurred. This

synchronized update ensures that all vehicles' positions are accurate and that the dynamics of the system evolve coherently.

The simulation runs for a predefined number of ticks, which corresponds to the total simulation time. Each tick updates the vehicles' state, and this process continues until the end of the simulation. The synchronization of all updates at each tick ensures that interactions between vehicles, such as car-following and lane changing, are consistently executed in series.

The time step  $dt$  is a key parameter in determining how frequently the system is updated. Smaller values of  $dt$  lead to a finer resolution and more frequent updates, resulting in a more detailed simulation but at the cost of higher computational load. On the other hand, larger values of  $dt$  reduce the computational cost but may introduce less accuracy in the modeling of vehicle behavior. Therefore, the choice of  $dt$  needs to strike a balance between accuracy and efficiency, depending on the needs of the simulation. We choose  $dt = 0.04$  seconds for our simulation, as the HighD dataset, which was used to obtain the LCD module data, has a time resolution of 0.04 seconds.

## 6.6.2 Simulation set-up

The simulation considers a three-lane roadway and operates with a time step ( $\Delta t$ ) of 0.1 seconds as we believe this should be fast enough to make accurate decisions. We consider eight simulation durations, with the simulation duration varying linearly between 30 seconds and 480 seconds. The vehicle setup includes an ego vehicle positioned in the centre lane and six other vehicles distributed across adjacent lanes. The ego vehicle starts at the origin with an initial velocity of 20 m/s (desired velocity of 30 m/s) and occupies the centre lane. Surrounding vehicles are initialized with randomized positions and velocities to simulate a realistic driving environment.

These vehicles are distributed across all three lanes, including vehicles in the same lane as the ego car and those in opposing and adjacent lanes. The randomized initial positions range from 2 to 100 meters from the ego car, while their velocities range from 5 to 30 m/s. Each vehicle is governed by an Intelligent Driver Model (IDM) for longitudinal control and a lane change model for lateral manoeuvres.

The execution of all the modules takes place in series every time instant. Each time instant, the car takes in the data from the car sensors and executes the LCD module and its enforcer, the LCE module and its enforcer and the CF module and its enforcer in series. Then, the results are passed to the controller to pass to the car, and the modules for the car to update its state begin the next step. The simulation is run in Python, but the enforcers are developed in C using the easyRTE tool.

To explore multiple scenarios, combinations of initial conditions and simulation durations are created using a Cartesian product of random seeds and duration intervals. We use ten randomised repetitions of the simulation durations, 80 simulations in total. The eight simulations with different simulation durations are parallelized using Python’s multiprocessing library, enabling efficient execution across multiple CPU cores.

As it may take a long time to observe undesired behaviour, we inject faults in the simulation to observe the behaviour of the AV and whether the enforcer takes corrective action or not in the presence of faults. We inject the following faults in the simulation:

- **Fault 1:** The LCD module outputs a lane change decision to the controller even when  $P_{LCD} = 0$  is not met. This is introduced by setting the LCD output to 1 when  $P_{LCD} = 0$  at multiples of time step 83.
- **Fault 2:** The car following distance becomes lesser than 2 metres. This is done by setting the gap to 1.99 metres at multiples of time step 89.
- **Fault 3:** The LCE module takes longer than 5 seconds for a lane change. This is done by artificially setting the lane change execution time to 5.04 seconds at multiples of time step 97.

## 6.7 Results

Once we obtain the coefficients for our mined properties and have the trained models and enforcers, we implement them in a Python simulation that mimics the ego/subject AV driving on a freeway. The results of the compositional training and the simulation are presented in this section.

### 6.7.1 Compositional Property Mining

As mentioned, we experiment with various threshold levels for property satisfaction to obtain our policies and models for the LCD module. We obtain different sets of policies for each threshold level. Equation 6.5 presents the mined policies for the best performing model in the simulations (i.e. threshold with the least number of faults).

$$\begin{aligned}
P_{LF} : lfX &> 80.77 + 11.23 * lfXVelocity + 9.72 * lfXAcceleration \\
P_{LB} : lpX &< -18.13 - 2.82 * lpXVelocity - 2.65 * lpXAcceleration \\
P_{RF} : rfX &> 16.00 - 3.21 * rfXVelocity - 1.51 * rfXAcceleration \\
P_{RB} : rpX &< -35.91 + 4.20 * rpXVelocity - 1.89 * rpXAcceleration \\
P_F : fX &> 0.45 + 31.14 * fXVelocity - 12.59 * fXAcceleration \\
P_B : pX &< -22.73 - 1.01 * pXVelocity + 4.52 * pXAcceleration
\end{aligned} \tag{6.5}$$

The given equations define conditions for positional properties (gaps) of a vehicle in relation to velocity and acceleration components for various directional states: Left Front (LF), Left Rear (LP), Right Front (RF), Right Rear (RP), Front (F), and Rear (P). These properties are then used to obtain  $P_{LCD}$  which is used by the LCD module enforcer to ensure that a lane change occurs only when adequate gaps are present between the cars.

We can observe in Equation 6.5 that half the policies have a greater than sign and half have a less than sign. This is because the relative positions of the cars are considered in the policies and the sign of the gap changes based on the position of the car relative to the subject vehicle (i.e., car - subject car). This also explains why the front car policies have positive intercepts (the first coefficient term) and the rear/back car policies have negative intercepts. Also, we can glean from the properties that the cars on average need more space on the front left owing to the large positive coefficients of  $P_{LF}$ . Other properties also show a similar behaviour.

## 6.7.2 LCD Model Performance

### Compositional LCD Model:

Table 6.2 shows the performance of all the LCD models. The first column in the table shows the LCD modules by the percentage satisfaction of  $P_{LCD}$ . *OLS* is the model with no optimisation and using the initial coefficients obtained by Ordinary Least Squares during the first step of the optimisation algorithm. *No Policy* model refers to the model trained normally without knowledge of the properties. The second column shows the mean percentage accuracy on the corresponding highD test dataset calculated over the five folds of the 5-fold cross-validation approach.

| Property Satisfaction Threshold (in %) | Compositional Mean Test Accuracy (in %) | Monolithic Mean Test Accuracy (in %) |
|----------------------------------------|-----------------------------------------|--------------------------------------|
| 90                                     | 82.96                                   | 87.16                                |
| 80                                     | 89.23                                   | 91.71                                |
| 70                                     | 89.64                                   | 92.00                                |
| OLS                                    | 97.57                                   | 98.69                                |
| No Policy                              | 66.79                                   | 71.92                                |

Table 6.2: Mean accuracies of the various LCD models identified by their satisfaction percentage threshold.

We can see from the table that it is easier for the model to learn to satisfy  $P_{LCD}$  if some form of the algorithm is used. Also, lower satisfaction percentages during the property mining stage yield higher accuracy of satisfaction post-training. This is expected as we notice that the number of property satisfying datapoints for training decrease as we decrease the threshold for property satisfaction. This creates a higher imbalance between the no lane change and lane change samples, making it easier for the model to learn the majority class and increasing accuracy.

#### Monolithic LCD Model:

From Table 6.2, we can see that the monolithic models are better at predicting correct lane changes from the highD dataset than the compositional models. As with the compositional models, the model performance increases as we decrease the threshold for property satisfaction. Nevertheless, the No Policy model has the lowest accuracy in both cases as it is the most balanced dataset (see Table 6.1) and this makes it difficult to learn from the real-life and complex highD dataset.

### 6.7.3 Enforcer Results: with compositional LCD module

Figure 6.12 shows how long each module takes to execute in every time step and how this execution time changes with increasing simulation time for the case where the LCD module threshold is set to 70%. The time reported by the bar plots is the mean time calculated across the ten repetitions. The line plots with the errorbars show the deviation from the mean.

We can see that the LCD module takes the longest time to execute as the ANN model is mathematically complex and needs a longer time to compute an output. It is interesting that this time exceeds the time it takes for Python to run the compiled enforcer files implemented in C. We do not report the time taken for other modules

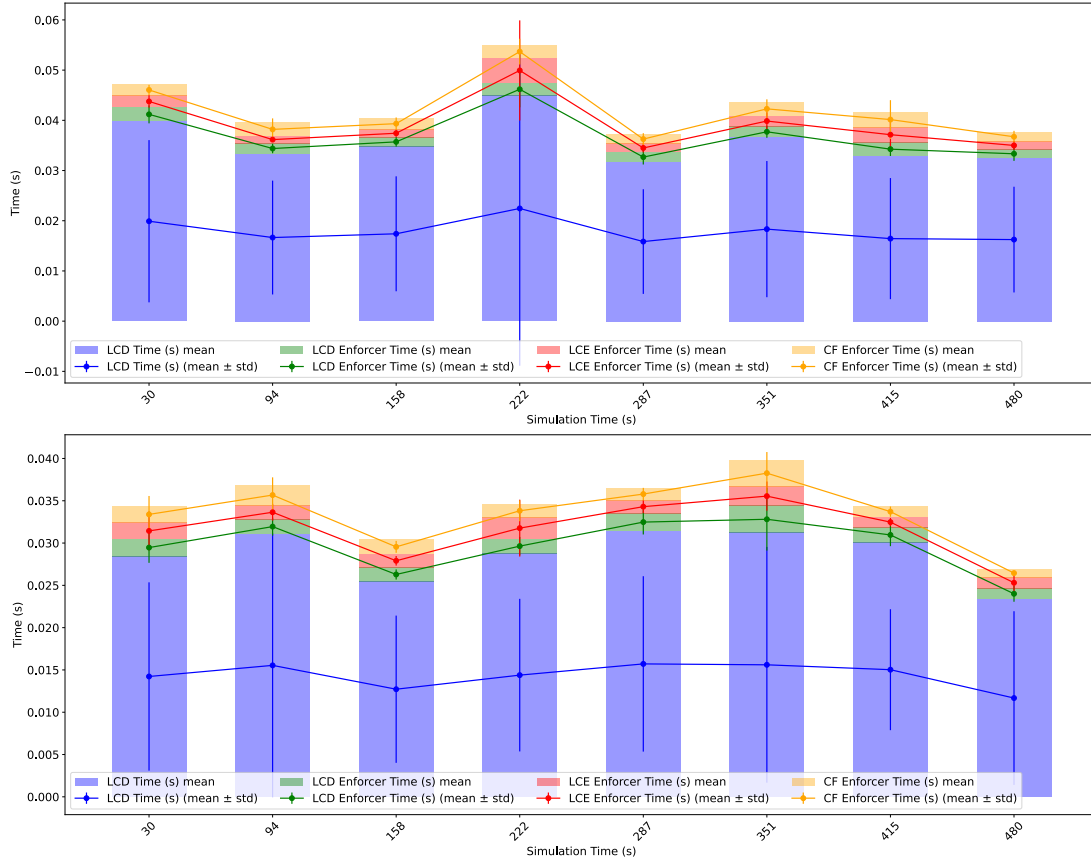


Figure 6.12: Mean Time taken by various modules in a single time step. The top plot corresponds to the system with LCD model built at 70% threshold, and the bottom plot corresponds to the case where no property is used to train the model.

(LCE and CF) as these modules are stateful, and it would be difficult to choose an arbitrary time instant to capture the execution time.

Although we see some variability in the execution time for LCD, there is no overall trend across the simulation durations, and the time required to run the enforcers is almost constant throughout. Nevertheless, we can see that the model trained on property (top plot) has a higher execution time than the one not trained on properties (bottom plots). The difference is negligible and could be caused either due to numerical artefacts or due to a simpler numerical structure of the LCD module. The enforcer times remain similar across the two plots.

Figure 6.13 shows the enforcer activations for all the thresholds considered for the LCD module. It shows the number of times the enforcer was activated (averaged across the ten repetitions), either due to faults injected artificially by the simulation or due to incorrect behaviour by the model.

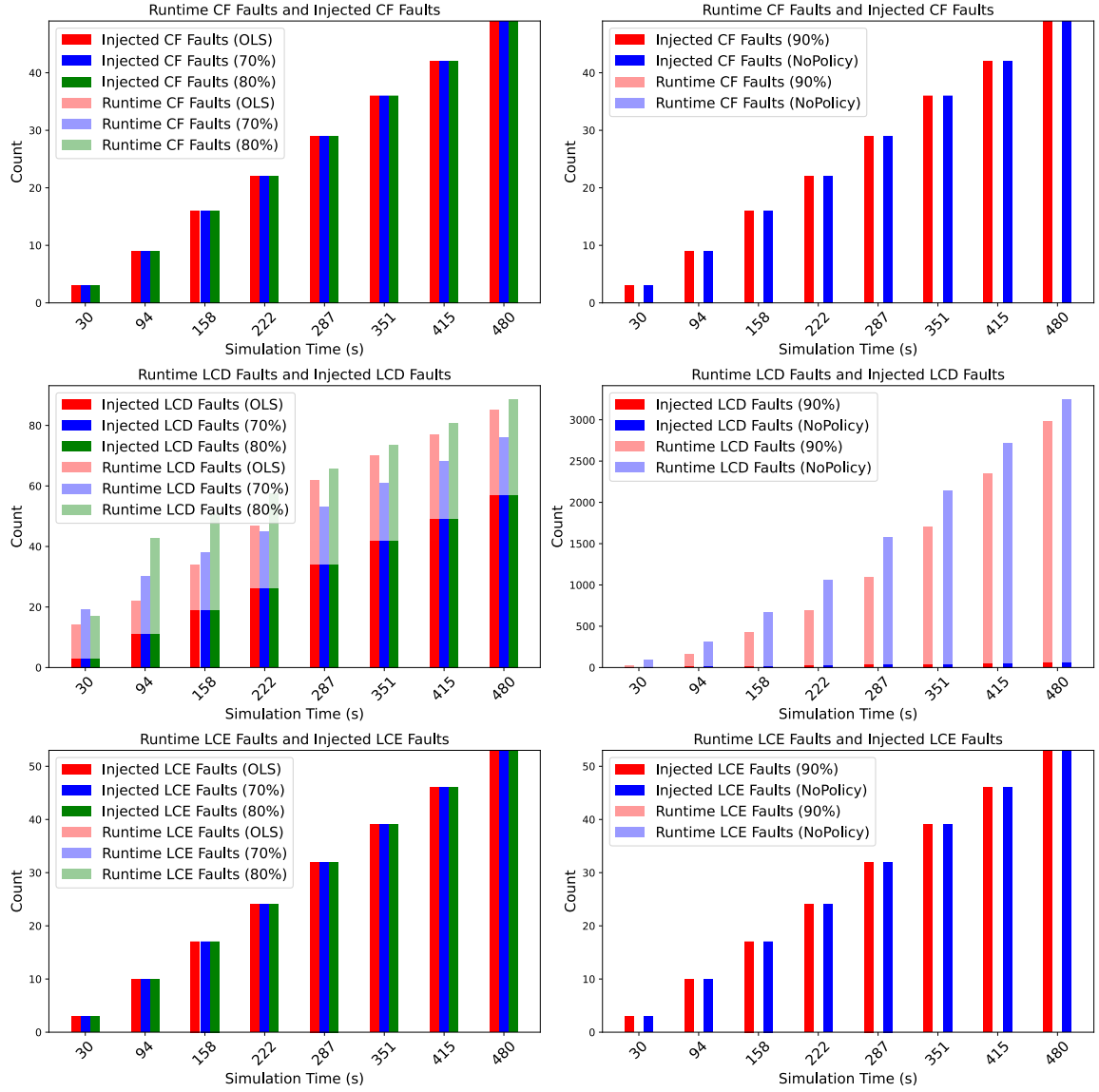


Figure 6.13: Enforcer activation results for all thresholds considered for the LCD module.

We can see from the figure that the CF and LCE modules do not produce any runtime faults, and the only faults their corresponding enforcers enforce are the injected faults. As these modules are not trained on property, we do not see any difference between the system with LCD module trained on property and the one where it is not. The LCD module, which is composed of ANN models, produced a lot of runtime faults that activated the LCD enforcer. The number of such faults was constant throughout the various simulation durations tried for the module trained on property.

In contrast, the runtime faults increase exponentially for the LCD module not trained using the property. Also, we see that the number of enforcer activations increases linearly with the linear increase in injected faults for all modules except for the LCD module not trained on property.

We can also see that the number of enforcer activations is low for thresholds 70%, 80% and OLS, whereas it is substantially higher for 90% and No Policy modules. This suggests that there is a trade-off between the number of enforcer activations and the property satisfaction threshold. In particular, it can be seen that too high of a threshold and too low of a threshold (No Policy threshold) can lead to more enforcer activations indicating that the model overfits to the training data and cannot generalise well if the threshold is too high or too low.

Overall, The 70% threshold seems to be the best trade-off between the number of enforcer activations and the property satisfaction threshold in our case. Moreover, in every instance where an enforcer was activated, the fault was handled and the car was observed to be safe.

#### **6.7.4 Enforcer Results: Comparison with monolithic LCD module**

Figure 6.12 shows how long each module takes to execute in every time step and how this execution time changes with increasing simulation time for the case where the LCD module is a monolithic model. The trend is similar to the compositional LCD module and the enforcer execution times are almost constant throughout the simulation duration. Nevertheless, the monolithic LCD module takes less time to execute than the compositional LCD module. This is expected as the monolithic model is a single model whereas the compositional model is a combination of three models spread across three subsequent stages.

Figure 6.15 shows the enforcer activations for all the thresholds considered for the monolithic LCD module. While the enforcement activation for the CF and LCE modules is the same as the compositional model, as there were no runtime faults, the LCD module behaviour is significantly different. The number of runtime enforcer activations is much higher for the monolithic LCD module than the compositional module, in general.

Particularly, the runtime faults for the monolithic model are an order of magnitude higher than all property-trained compositional modules. In contrast to the compositional module, the runtime faults in the monolithic model is higher for the model

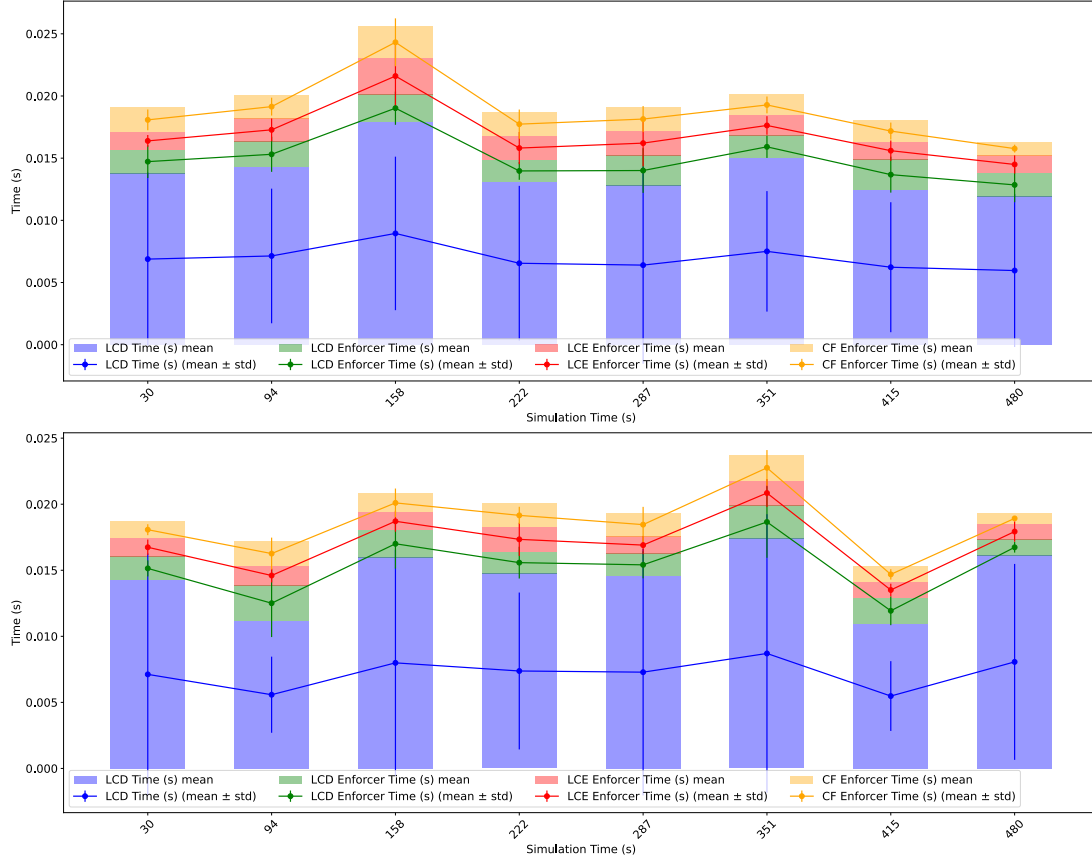


Figure 6.14: Mean Time taken by various modules in a single time step. The top plot corresponds to the model built on 70%  $P_{LCD}$  satisfaction, and the bottom plot corresponds to the case where no property is used to train the model.

trained on property than the one not trained on property. Also, the runtime faults increase with simulation time, exponentially in the case the model is trained on property. It appears that the monolithic model overfits to the training data a lot more and fails to properly generalise when exposed to data from realistic simulations.

These results suggest that the compositional models are better suited for the compositional policy enforcement discussed in this chapter, even though they take slightly more time to execute than the monolithic simulation. Overall, the compositional models trained on property seem better suited for the AV case study.

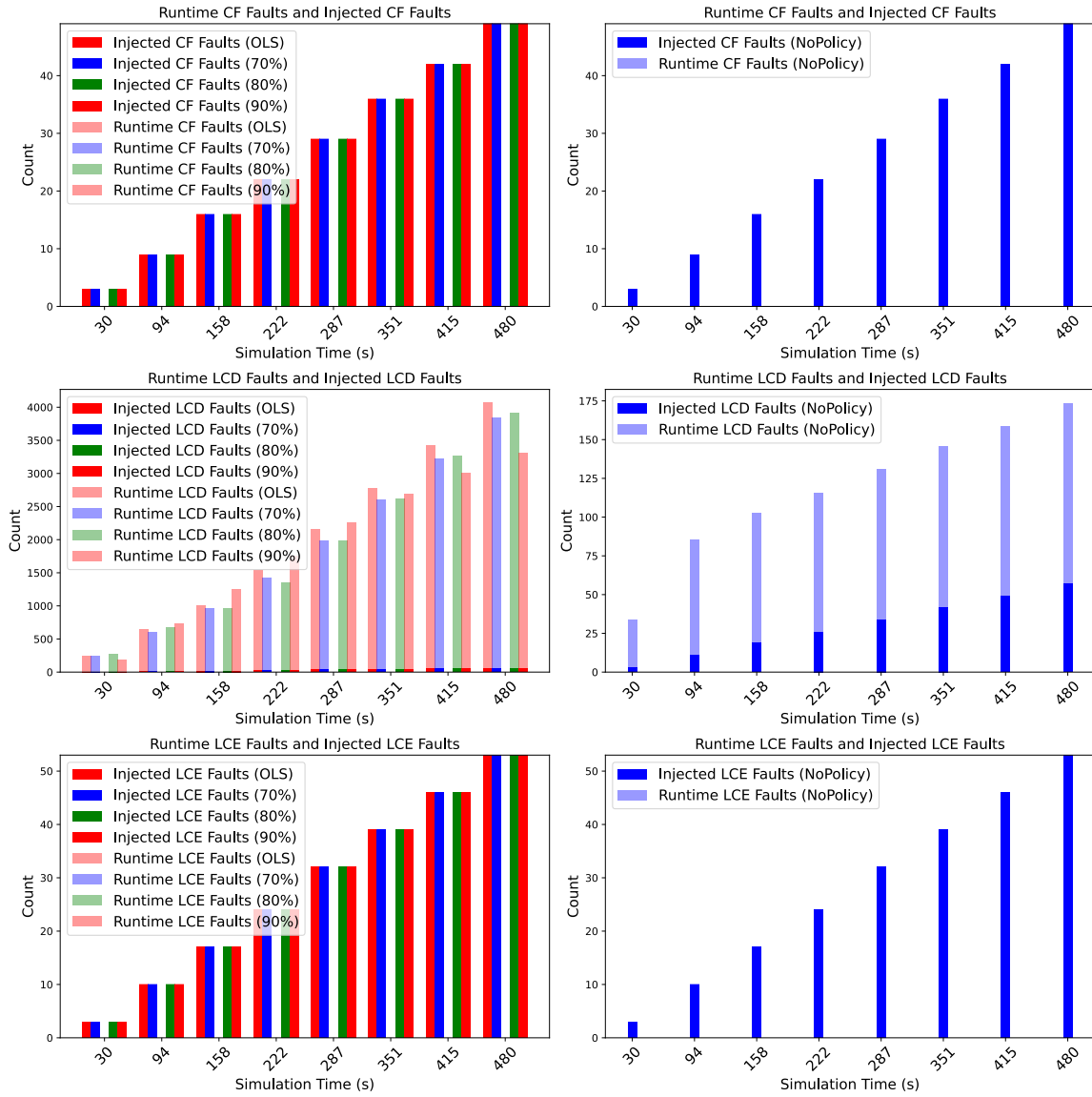


Figure 6.15: Enforcer activation results. The left column is from the simulation with LCD model built on 70%  $P_{LCD}$  satisfaction and the right column is from the simulation with LCD model not trained on property.